The goal of the second exercise is to create multiple coordinated views on web pages, where users can interactively explore the data. The data processing is done in the back-end with Python (using a Flask server). This is a very lightweight method to combine the power of data processing of Python with the versatile interactive visualization abilities of d3.

We provide a scaffold, where the client can request some dummy data from the server and the world map is rendered. For installation instructions, **please consult the README.md** from the framework. For useful links and some hints, **please check the resources at the end of this section** (below the image). You can and should generate multiple JavaScript source files.

The exercise is split into several sub-tasks:

Task 1 (2 points): Load the CSV file generated in Exercise 1 on the server, Filter the data to the countries in the list **COUNTRIES**, and return the data to the client. We use the Flask template engine Jinja 2 for communication. For data loading, we recommend using Pandas and JSON.

COUNTRIES: ['Afghanistan', 'Albania', 'Algeria', 'Angola', 'Argentina', 'Armenia', 'Australia', 'Austria', 'Azerbaijan', 'Brazil', 'Bulgaria', 'Cameroon', 'Chile', 'China', 'Colombia', 'Croatia', 'Cuba', 'Cyprus', 'Czech Republic', 'Ecuador', 'Egypt, Arab Rep.', 'Eritrea', 'Ethiopia', 'France', 'Germany', 'Ghana', 'Greece', 'India', 'Indonesia', 'Iran, Islamic Rep.', 'Iraq', 'Ireland', 'Italy', 'Japan', 'Jordan', 'Kazakhstan', 'Kenya', 'Lebanon', 'Malta', 'Mexico', 'Morocco', 'Pakistan', 'Peru', 'Philippines', 'Russian Federation', 'Syrian Arab Republic', 'Tunisia', 'Turkey', 'Ukraine']

Task 2 (2 points): On the server, compute a PCA based on the data of the most recent year only. You can use sci-kit learn to do that. Note that you need to take care of scaling the features appropriately! Send the resulting 2D coordinates (and potential intermediate steps required for visualization) to the client.

Task 3 (2 points): Use PCA result as a 2D scatterplot, where individual dots represent the countries. The association of the individual dots with the countries has to be clear.

Task 4 (4 points): Render a world map showing identifying maps of countries by coloring them with the appropriate color scheme. Allow the selection of the map rendered

Task 5 (5 points): Link the two charts:

- By hovering over a dot in the PCA scatterplot, highlight the corresponding country on the map. **(up to 1.5 points)**
- By hovering over a country on the map, highlight the corresponding dot in the scatterplot. **(up to 1.5 points)**
- By clicking on a country on the map, render a line plot to show the change in the variable value of that country from 1960 – 2020. **(up to 2 points)**

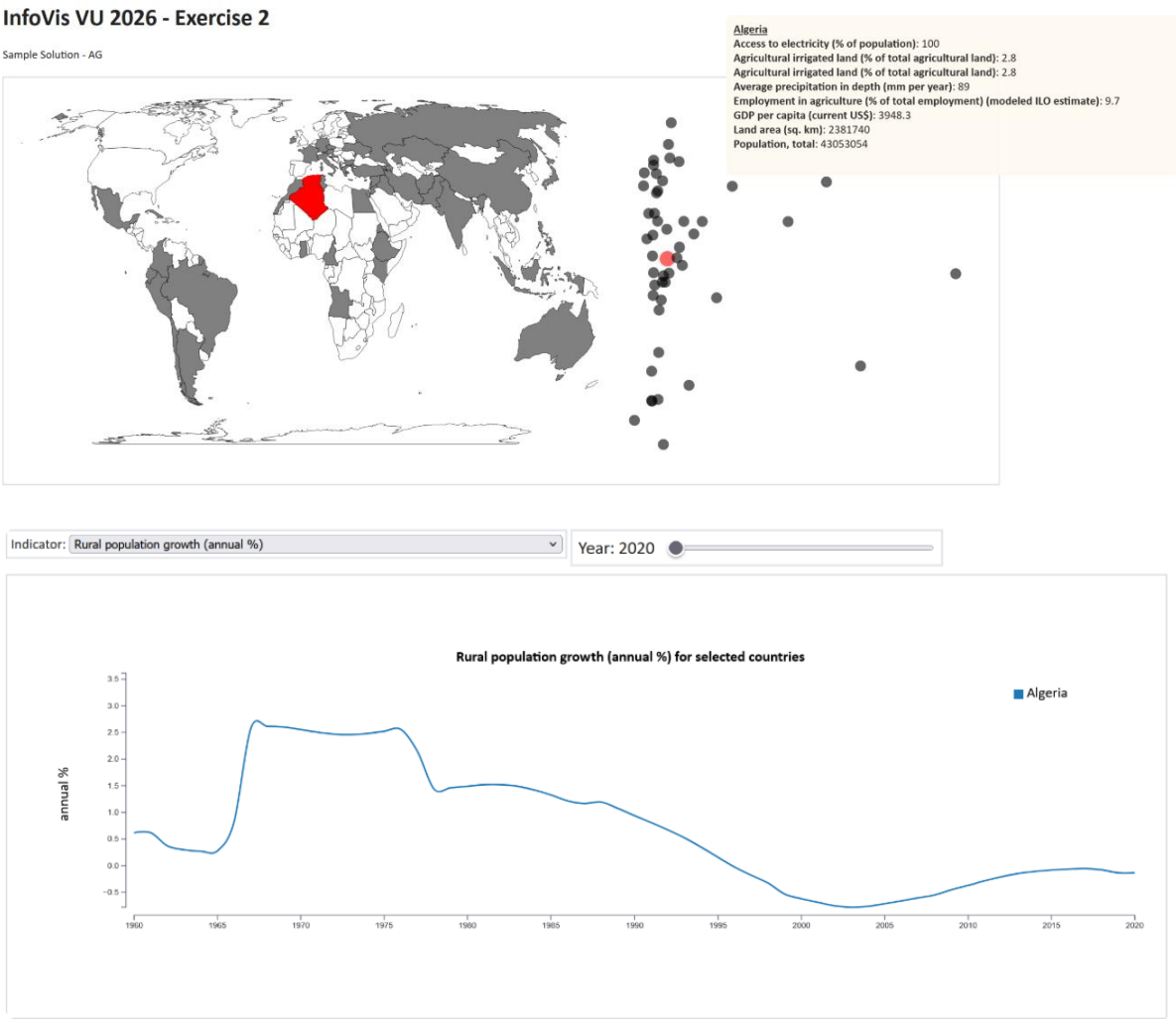
Task 6 (3 points): Extend your system to support fully coordinated views using a shared interaction state across the PCA scatterplot, choropleth map, and time series.

- Add rectangular brushing to the PCA scatterplot using d3.brush. Countries inside the brush selection must be highlighted in the scatterplot and on the map. The time series view must update to display all brushed countries. Clearing the brush must reset the selection. **(up to 1 point)**
- Add a year slider for temporal interaction. Changing the selected year must update the choropleth map and the time series accordingly. **(up to 1 point)**

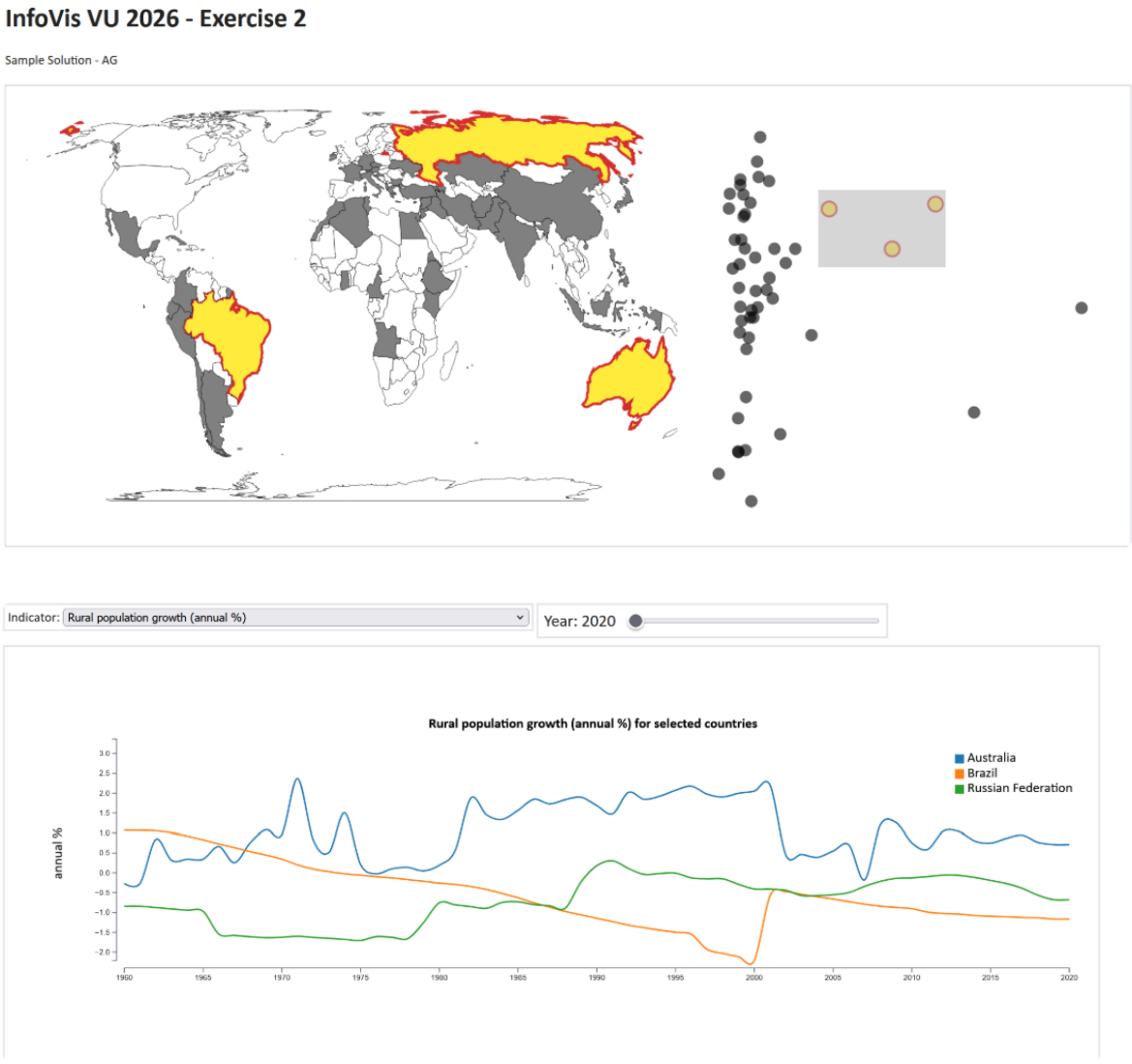
- Ensure attribute-based coordination. Changing the selected indicator must consistently update the choropleth coloring, the time series values, and the styling of the scatterplot. All updates must use d3's enter/update pattern and charts must not be fully redrawn. **(up to 1 point)**

Bonus Task (2 extra points): On hovering over a country on the Map, show details-on-demand (e.g., a tooltip showing 8 variable values of the country for the most recent year).

Here is an example of how the interface could look when hovering and clicking on the map of "Algeria" and when selecting the "Rural Population growth (annual %)" indicator from the drop-down menu.



The following screenshots illustrate example interaction states of the interface, including coordinated highlighting, brushing, tooltip display, year selection, and time series updates.



There are many online d3 tutorials. Here is a small selection:

- Tutorials directly on d3 Github
- Tutorialspoint
- d3 wiki
- d3.v4 Tutorial by Square

There are numerous d3 implementations of standard charts out there. You can look at those existing implementations and adapt them to your needs, for example:

- Scatterplots in d3
- Heatmap tables in d3

Other useful resources:

- General introduction to flask + d3: https://benalexkeen.com/creating-graphs-using-flask-and-d3/

- About axes in d3
- Color maps in d3

Note that we are using d3.v5, which is no longer compatible to the more wide-spread d3.v3 used for many online examples!

Some common mistakes that should be avoided and some hints:

- A common mistake is that charts are not properly updated but rather entirely removed and drawn from scratch after each user interaction (or drawn on top of the previous chart). However, this can be far too slow for interactivity. Familiarize yourself with d3's enter / update mechanism. You can look into the d3 tutorial slides provided below.
- Everything you need to know about the data is included in the data. There is no need to hard-code country names or indicator names. You may want to have a look at d3-collections for data manipulation techniques.
- Note that it is possible to pass multiple named variables to the render template.